Lab 9- EEPROM

-Purpose: Investigate the PIC16F883 data EEPROM: read and write stored data, verify retention across power cycles, measure operation timing, and apply reliable nonvolatile storage in embedded systems.

-Standards and References:
- RCET 3375 Lab Standard: Lab book, SFR, circuit, measurement, and submission requirements.
- RCET 3375 PIC-AS Style Guide and PIC-AS project setup.
- PIC Datasheet (DS40001291H).
- Section 10.1, pages 113-115: EEPROM read/write operations and control registers
- Section 10.3.1, pg 119: endurance and refresh
- Section 17.5, pg 246: EEPROM endurance, retention, and write time specifications.
- DLG7137 display data sheet: display pinout and timing
- Lab 2 - PIC I/O: previously developed display and input interfaces.

-Equipment and materials:
- PIC16F883 circuit with 4 MHz crystal and suitable power supply.
- PICkit 3 or equivalent, MPLAB X, and pic-as
- DLG7137 single-character display
- oscilloscope, logic analyzer, frequency counter, and digital multi-meter as needed
- Lab book, breadboard, and various components as needed.

-Part 1 - EEPROM Persistance Proof of life:
- Goal: Select A or B in RAM, deliberately save it to data EEPROM address 0x00, and continuously display the stored EEPROM value. Measure read/write timing and verify retention after a power cycle.

- Required program behavior:
  - Buttons: Use two active-LOW buttons with internal weak pull ups the Selected button (RB0) and Write button (RB1). Debounce both pressing and releasing each button in software.
- Choose a value: Normal display: When the write button is held, display the pending RAM selection, initially A. Each valid Select button press toggles the selection between A and B. Pressing the select button while the Write button is released leaves the selection unchanged.
  - Normal display: When the Write button is released, continuously read EEPROM address 0x00 in the main loop and display the stored A, B, or ? for any other value.
  - Save a value: A valid Write button press enters selection mode. On its debounced release, save the pending RAM value to EEPROM once, wait for the EEPROM operation to finish, and resume displaying the stored value.
  - Write discipline: An EEPROM write may start only after a complete, debounced Write button press-and-release cycle. Do not write at startup, during normal main-loop reading, when the Select button button is pressed, or repeatedly while the Write button is held. Contact bounce on either button must not produce an extra write.
  - Timing diagnostic: RA1 is high only during each EEPROM read, and RA2 is high only during the EEPROM write operation, including the wait for completion. Keep both LOW otherwise. Exclude display-update time from the read diagnostic and preserve the uninterrupted.

10/19/2026
LRG

Lab 9 - EEPROM

Before Lab:

- the PIC16F883 EEPROM SFR maps and read/write produce, including the protected write sequence, write-complete indication, and interrupt precautions
- the circuit schematic and electrical loading checks for both buttons, display, and diagnostic.
- flow charts for the main loop, Select button handling. Write button handling, software debouncing, EEPROM read/write, and display update.
- predicted display and diagnostic behavior for unrecognized EEPROM data, selection without saving, saving on Write button release, contact bounce, and power cycling.

- EEPROM durability: From Section 17.5, document the minimum and typical byte endurance for both temp ranges (D120/D120A), typical and maximum write time (D122), characteristic data retention (D123), and total EEPROM writes before refresh (D124). Explain endurance, retention, and refresh using PIC16 Section 10.3.1.

- Endurance calculation: Suppose a programming error repeatedly writes the same EEPROM address, starting another write as soon as the previous one finishes. Estimate the time to consume the minimum specified byte-endurance budget using the typical and maximum write time for both temp ranges.

- The calculation is:

$$T_{budget} = N_{writes} \, t_{write}$$

Worked example (-40°C to +85°C, typical write time): Multiply the minimum byte endurance by the typical time per write to find the elapsed time, then converts second to minutes.

$$T_{budget} = 100{,}000 \text{ writes} \times 5 \frac{ms}{write}$$

$$500{,}000 \text{ ms}$$

$$\frac{500{,}000}{1000} s \rightarrow T_{budget} = 500s \text{ or } 8min \ 20s$$

- Complete the remaining three combinations using the maximum write time and the extended-temp endurance value. Show all substitutions, arithmetic, conversions, and units. State which software overhead you ignored, and explain why the endurance specification is a planning limit rather than an exact failure point. A mistaken EEPROM write inside the PIC16 main loop could use up the specified endurance budget in minutes and permanently damage that address. Verify that your code cannot write repeatedly before leaving it running. Do not perform a destructive endurance test.

In the Lab:

1) Build the two-button and DLG7137 circuit with RA1 and RA2 available for oscilloscope measurements.

2) Connect all four oscilloscope channels: CH1 / Select button, CH2 / Write button, CH3/EEPROM read diagnostic (RA1), CH4/EEPROM write diagnostic (RA2).

3) Verify continuous EEPROM reading with the Write button released. Press and hold the Write button to display the RAM selection, then use the Select button to toggle between A & B. Confirm that CH4 stays low through selection.

4) Trigger on both falling and rising edges of the Select button (CH1). Verify that its press and release do not produce an EEPROM write, whether or not the write button is held.

5) PIC16

10/19/2026
LRG

5) Trigger on both falling and rising edges of the Write button (CH2). Verify that the falling edge (press) starts selection mode without writing, and that only a valid debounced rising edge (release) leads to one CH4 write pulse. Include sufficient post-trigger time for software debouncing and EEPROM programming.

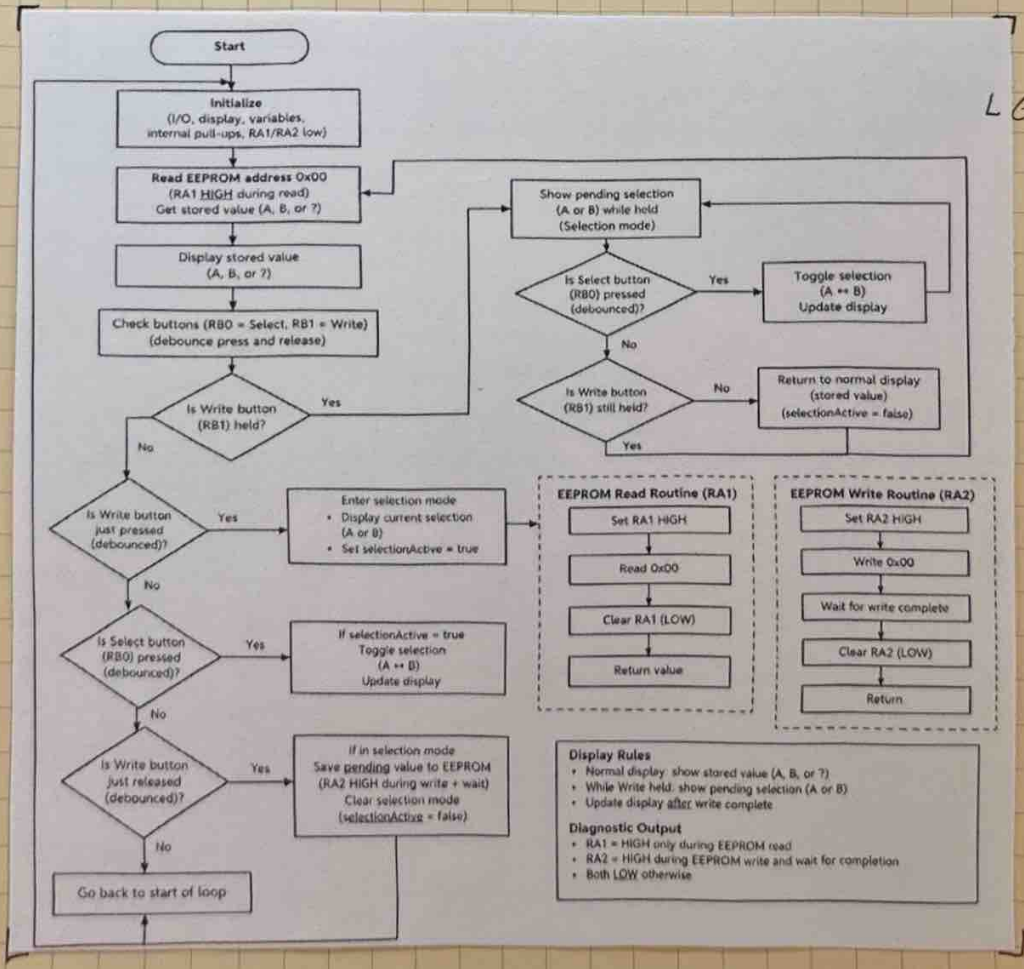
6) Record labeled four channel captures showing these edge-trigger tests, including continous CH3 activity, contact bounce, and the CH4 response. Use sufficient pre-trigger time to show selection before the Write button release; use a separate expanded acquisition to measure the short read pulse when needed.

7) Measure and label both EEPROM read and write pulse widths in the scope captures. Compare write duration against the data sheet and account for diagnostic/software overhead.

8) Test bold long holds, repeated Select button presses, and physical press/release bounce. Verify that each complete Write button press-and-release produces exactly one write and the Select button alone can't write.

9) Save A, remove all power, and restore it without touching either button or reprogramming. Verify the stored character appears. Repeat for B.

10) Inspect the EEPROM registers and write-complete indication in the debugger. Make oscilloscope timing measurements during uninterrupted program execution.

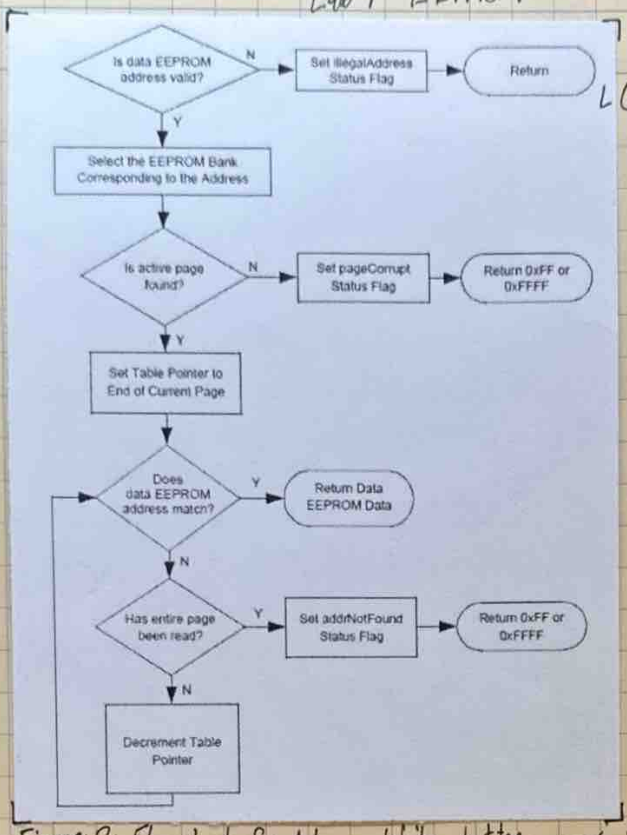


LG

Figure 1: EEPROM Persistence Proof of Life flow chart

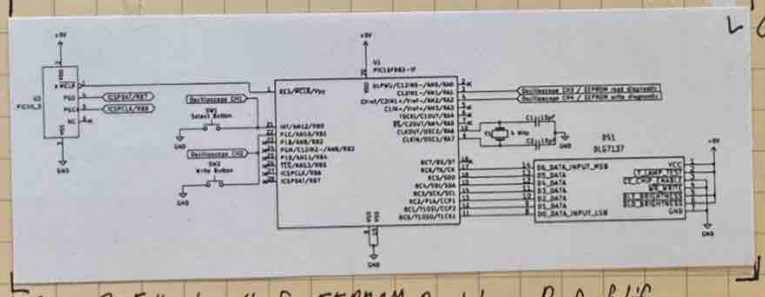
10/23/2026
LRG

Lab 9 - EEPROM



LG

Figure 2: Flowchart for debounce Write button press/release cycle to EEPROM once.



LG

Figure 3: Full schematic for EEPROM Persistance Proof of life

```
Setup:
    ;       Bank 3
    BANKSEL ANSEL
    CLRF    ANSEL           ;make every pin a normal digital pin
    CLRF    ANSELH          ;RB0 and RB1 can also be analog pins, so this matters
    ;       Bank 1
    BANKSEL OPTION_REG
    BCF     OPTION_REG,7    ;turn on the built-in pull-up resistors for PORTB
    MOVLW   0b00000011
    MOVWF   WPUB            ;use the pull-ups on RB0 and RB1 only
    BSF     TRISB,0         ;RB0 is an input (Select button)
    BSF     TRISB,1         ;RB1 is an input (Write button)
    BCF     TRISA,1         ;RA1 is an output (read marker)
    BCF     TRISA,2         ;RA2 is an output (write marker)
    CLRF    TRISC           ;all of PORTC is an output (the display)
    ;       Bank 0
    BANKSEL PORTA
    CLRF    PORTA           ;both markers start LOW
    CLRF    PORTC           ;display starts blank
    MOVLW   0x41
    MOVWF   Pending         ;the pick starts out as 'A'
```

LG

Figure 4: Setup SFR's for EEPROM

10/20/2026
LRG

```
; MAIN LOOP: read the stored letter and show it, over and over.
; If the Write button is pressed, go and pick a new letter instead.
Main:
    CALL    EepromRead    ;the stored value comes back in W
    MOVWF   Display
    XORLW   0x41          ;is it A
    BTFSC   STATUS,2
    GOTO    ShowStored    ;if yes show it
    MOVF    Display,W
    XORLW   0x42          ;is it B
    BTFSC   STATUS,2
    GOTO    ShowStored    ;if yes show it
    MOVLW   0x3F          ;it is neither, so show a question mark
    MOVWF   Display
ShowStored:
    MOVF    Display,W
    CALL    DisplayLetter ;put the letter on the display

    BTFSC   PORTB,1       ;is the Write button pressed? (LOW = pressed)
    GOTO    Main          ;if not keep reading
    CALL    Wait10ms      ;if yes then wait 10 ms to make sure it is not just bounce
    BTFSC   PORTB,1       ;verify that it is still pressed
    GOTO    Main          ;if not it was only bounce, so ignore it
```

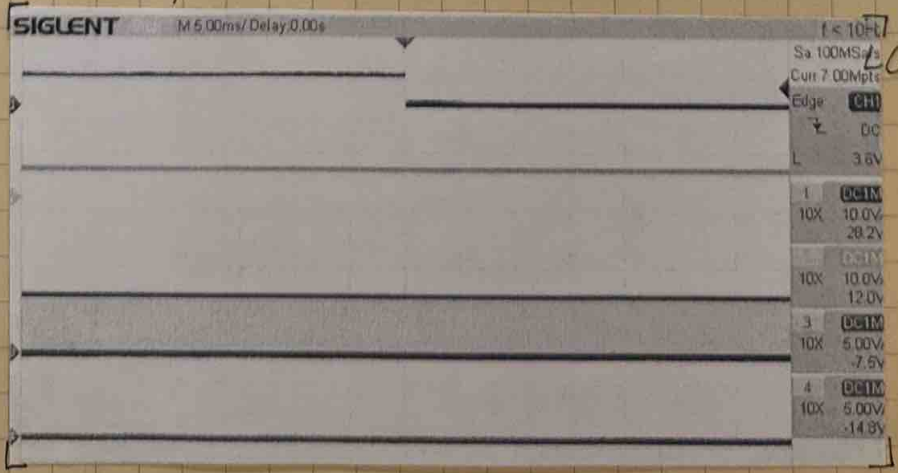
LG

•Figure 5: Main Code for EEPROM and display Letter saved.

```
;PICK A LETTER: we stay here while the Write button is held down.
;The display shows the pick. The EEPROM is NOT touched in here.
Selecting:
    MOVF    Pending,W
    CALL    DisplayLetter ;show the letter we are picking
    BTFSS   PORTB,1       ;has the Write button been let go? (HIGH = released)
    GOTO    CheckSelect   ;if not still held then look at the Select button
    CALL    Wait10ms      ;if it looks released then wait 10 ms to make sure
    BTFSC   PORTB,1       ;is it still released
    GOTO    SaveIt        ;yes, it is a real release, so save the letter
                          ;no, it bounced back down, so carry on picking
CheckSelect:
    BTFSC   PORTB,0       ;is the Select button pressed? (LOW = pressed)
    GOTO    Selecting     ;if no keep waiting
    CALL    Wait10ms      ;if yes then wait 10 ms to make sure it is not bounce
    BTFSC   PORTB,0       ;is it still pressed?
    GOTO    Selecting     ;if no then it was only bounce, so ignore it
    MOVLW   0x03
    XORWF   Pending,F     ;switch the pick: 'A' becomes 'B', 'B' becomes 'A'
    MOVF    Pending,W
    CALL    DisplayLetter ;show the new pick right away
WaitSelectUp:
    BTFSS   PORTB,0       ;wait here until the Select button is let go
    GOTO    WaitSelectUp
    CALL    Wait10ms      ;let any bounce on the release settle
    GOTO    Selecting
```

LG

•Figure 6: Display and Check for select button



LG

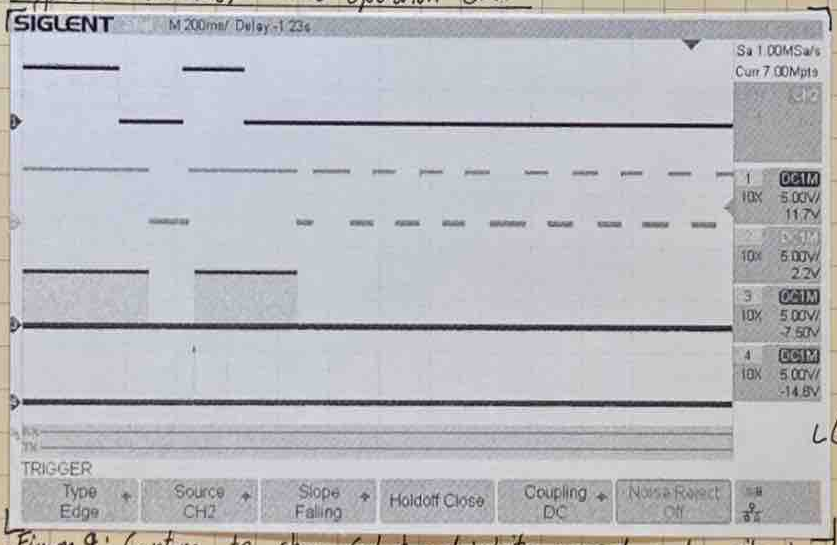
•Figure 7: Falling edge trigure for Select button

Lab 9 - EEPROM

10/20/2026
LRG

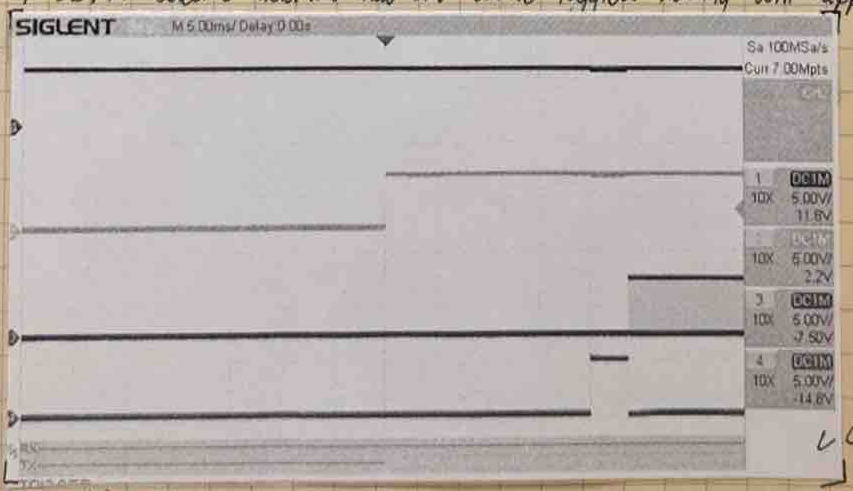
```
;EepromWrite: stores 'pending' at address 0 and waits until the chip
;says it is finished. RA2 is HIGH the whole time.
EepromWrite:
    BSF     PORTA,2         ;write marker ON
    BANKSEL EEADR
    CLRF    EEADR           ;the address to write is 0
    MOVF    Pending,W
    MOVWF   EEDATA          ;the letter to store
    BANKSEL EECON1
    BCF     EECON1,7        ;pick data EEPROM
    BSF     EECON1,2        ;allow writing
    BCF     INTCON,7        ;make sure no interrupt can get in the way of the next lines
    MOVLW   0x55            ;the chip needs this exact code, in this exact order,
    MOVWF   EECON2          ;with nothing in between, or it ignores the write
    MOVLW   0xAA
    MOVWF   EECON2
    BSF     EECON1,1        ;start the write
WaitWrite:
    BTFSC   EECON1,1        ;the chip clears this bit when it is finished
    GOTO    WaitWrite       ;if it is still writing then wait
    BCF     EECON1,2        ;turn writing off again so nothing can write by accident
    BANKSEL PIR2
    BCF     PIR2,4          ;clear the "write finished" flag
    BANKSEL PORTA
    BCF     PORTA,2         ;write marker OFF
    RETURN
```

-Figure 8: EEPROM write operation Code



-Figure 9: Capture to show Select and Write pressed and write toggled

-The write only happens after select and write are pressed together to toggle the letters A & B. If Select is held low ~~had~~ LG and write toggled nothing will happen.



-Figure 10: Write rising edge and write diagnostic set before rereading

TJR
10-20-26

10/20/2026
LRG

Part 2 - Reusable EEPROM Read/Write Routines and Timed Section

Goal: Extend Part 1 with reusable EEPROM routines, a non-blocking write, and a seperate verification routine. Replace hold-to-select with a timed selection mode.

Pin	Direction	Function
RA0	Output	Selection-mode LED, HIGH during selection
RA1	Output	Main-loop diagnostic pulse, once per iteration
RA2	Output	EEPROM error LED, HIGH when `EEPROM_ERROR` is set
RB0	Input	Select button, active LOW with internal pull-up
RB1	Input	Write button, active LOW with internal pull-up

LG

Table 1: I/O assignments for Part 2 & DLG7137 connection on PORTC stay the same

GPR	Purpose
ee_address	EEPROM byte address for read or write
ee_data	Byte returned by a read or written during a write
ee_status	`EEPROM_READ`, `EEPROM_WRITE`, and `EEPROM_ERROR` status flags

Table 2: EEPROM routines for GPR RAM defining 3 status flags with the EQU

-The EEPROM subroutines must use only these three RAM GPR's and work without modification in other programs. The calling application may use additional RAM GPRs. You may choose whether to use polling or the EEPROM write-complete interrupt to determine when a write finishes. If using the interrupt, the ISR should call EEPROMVerifyWrite once per completed write. You may use internal helper subroutines for EEPROM read/write, but call them only from the EEPROM routines and document the maximum stack depth.

-Use these routines labels for external calls:

- EEPROMRead: Load ee_address before the call. Set the EEPROM_READ status flag on entry, place the EEPROM byte in ee_data, and clear the EEPROM_READ status flag before returning.
- EEPROMWrite: Load ee_address and ee_data before the call. If EEPROM_Write or EEIF is set, return immediately without changing ee_status. Otherwise, compare the stored byte at ee_address with the requested value in ee_data without overwriting ee_data. If the values match, return without writing. Preserve any existing EEPROM_ERROR status flag. Otherwise, set the EEPROM_WRITE status flag, start the protected write sequence, and return without waiting for the write to finish. Keep the unlock seqence uninterrupted and follow the interrupt precautions from Part 2. Before returning, ~~ste~~ restore GIE to its previous state, enabling interrupt only if they were enabled on entry. You may use a bit in ee_status to save the previous GIE state; clear that bit after restoring GIE.
- EEPROMVerifyWrite: Call after hardwear writes finishes. Compare the byte at ee_address with the expected value still in ee_data. Don't overwrite ee_data during the comparison. If they match, clear ee_status. If they differ, clear the EEPROM_WRITE status flag and set the EEPROM_ERROR status flag. Clear EEIF and WREN before returning.

- Button Service:
  - Service the Select and Write buttons by polling or interrupts. Keep the Part 1 button debouncing and accept one action per press.
  - Do not start another EEPROM operation or change ee_address or ee_data while is awaiting verification.
  - Ignore button events while either the EEPROM_READ or EEPROM_WRITE status flags are set. The EEPROM_EEIF & ERROR status flags are set. The ~~EEPROM_ERROR~~ doesn't block the buttons.

10/20/2026
LRG

Lab 9 - EEPROM

•Timed selection:

•Normal operation: RA0 is off. Display the character stored at EEPROM address 0x00.

•First Selection button (RB0) press: Enter selection mode, select A, display A, turn RA0 ON, and start an approximately one second timeout.

•Additional Select button press: Toggle the pending value between A and B, update the display, and restart the one-second timeout.

•Timeout: About one second after the last Select button press, exit selection mode, turn RA0 off, and resume the EEPROM stored value display. A timeout doesn't start an EEPROM write.

•Write button (RB1): A debounced press before the timeout exits selections mode, stops the selection timer, turns RA0 off, and calls EEPROMWrite to save the pending character to address 0x00. Ignore Write button presses outside selection mode.

•Optional address-testing tip: In the application, define current_address with EQU as a RAM GPR and initialize it to 0x00. Use its value when loading ee_address for display reads and saved writes. Changing this one value makes it easier to test other EEPROM addresses. This is optional and is not part of the reusable EEPROM routines.

•Main-loop behavior: Repeat these steps in MainLoop:

1) Pulse RA1: Set and immediately clear the main-loop diagnostic pin.

2) Check write completion: If polling, check EEIF while the EEPROM_WRITE status flag is set and call EEPROMVerifyWrite when the write finishes. If using the EEPROM write-complete interrupt, the ISR calls EEPROMVerifyWrite instead.

3) Updated RA2: Set the error LED to match the EEPROM_ERROR status flag on every iteration.

4) Service timeout, then buttons: Check the selection timeout before accepting button presses. Poll the buttons or process "button events supplied by interrupts.

5) Update the DLG7137 display: Show the pending character while in selection mode. Otherwise, read EEPROM address 0x00 and display A, B, and ? for any other stored value. While the EEPROM_WRITE status flag is set, keep displaying the last character without reading EEPROM.

6) Repeat...

•Before Lab:

-Document the three GPR assignments, status flag bit positions, inputs and outputs of each EEPROM routine, and how GIE is saved and restored during the protected write sequence. Prepare a state diagram for selection, timeout, writing, verification, and errors. Include the one second timing plan and address/value test table. Document the RA0 and RA2 LED connections and loading, and the RA1 diagnostic pin. Explain why EEPROM reads do not consume write endurance and why checking for an unchanged value reduces wear.

LG

10/20/2026
LRG

Lab 9 - EEPROM

In the Lab:

1) Refactor the Part 1 EEPROM code into the three named routines. Test several byte values at different EEPROM addresses and check them again after removing and restoring power.

2) Check the EEPROM_READ status flag on entry to and exit from EEPROMRead. Confirm the returned byte is in ee_data.

3) Confirm EEPROMWrite returns with the EEPROM_WRITE status flag set before hardware programming finishes. Use RA1 on the oscilloscope to show main continuing to run during the write.

4) Handle EEIF and call EEPROMVerifyWrite once per completed write. Confirm that successful verification clears ee_status. Test the button are ignored while the EEPROM_READ or EEPROM_WRITE status flag is set.

5) Test verification failure by deliberately programming a different byte from the expected value retained in ee_data. Confirm that EEPROM_ERROR is set and the RA2 LED turns on. Restore the code, make a successful changed-value write, and confirm the error clears. Do not try to cause an actual EEPROM failure or run an endurance test

6) Try saving the value already stored. Confirm no EEPROM write starts and any previous EEPROM_ERROR status flag remains unchanged.

7) Measure the selection timeout. Check that the first Select press shows A, later toggles A/B and restart the timer, and timeout exits without writing. Confirm that Write is ignored outside selection mode, but exits selection mode immediately when accepted. Verify that the selection timer doesn't affect a pending write and that EEPROM_ERROR alone doesn't block the buttons.

8) Capture RA1 during normal operation and an EEPROM write. Check that RA2 follows the EEPROM_ERROR status flag on every main-loop iteration.

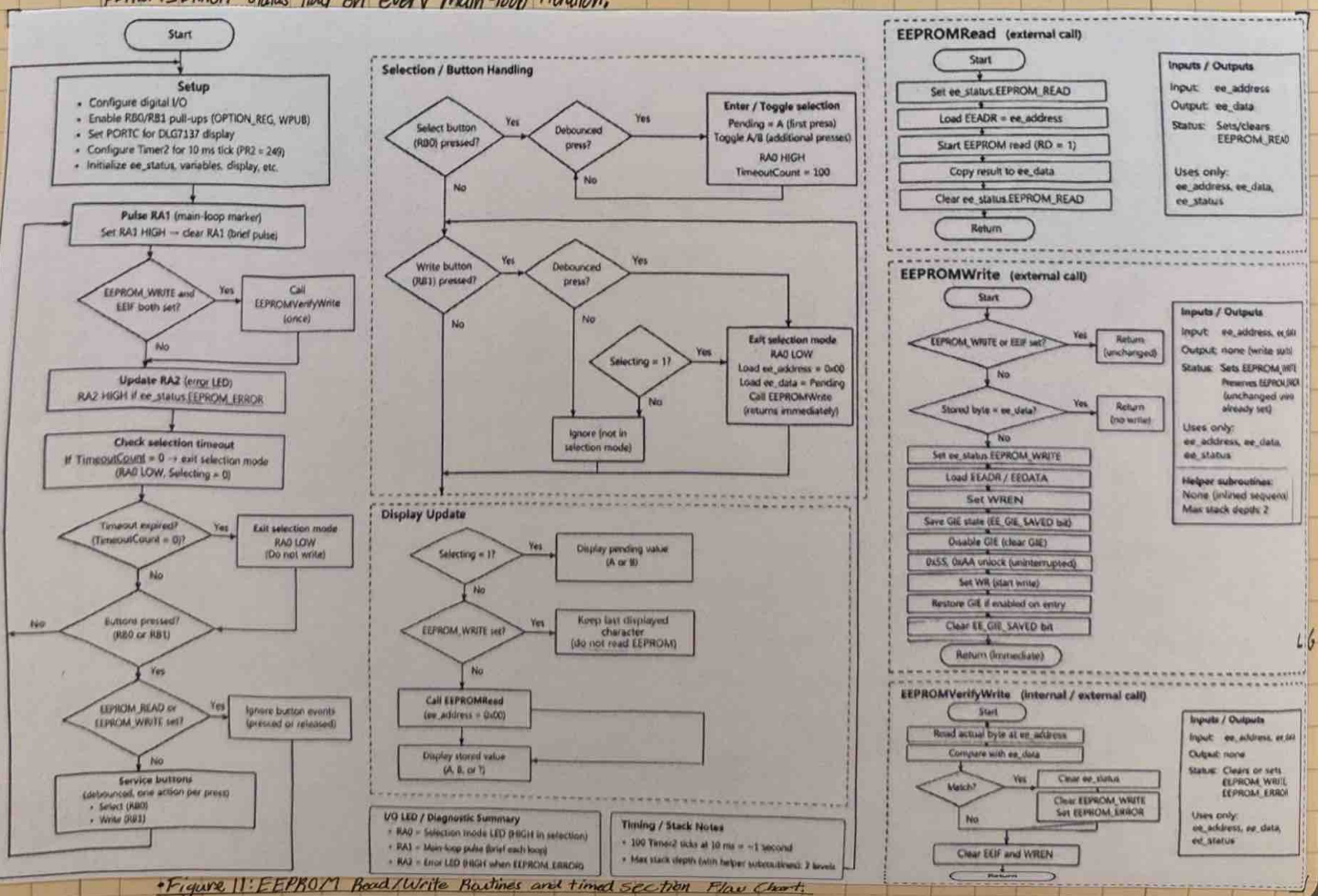


Figure 11: EEPROM Read/Write Routines and timed section Flow Chart.

10/21/2026
LRG

Lab 9 - EEPROM

```
Setup:
    ;       Bank 3
    BANKSEL ANSEL
    CLRF    ANSEL           ;make every pin a normal digital pin
    CLRF    ANSELH                                                  LG
    ;       Bank 1
    BANKSEL OPTION_REG
    BCF     OPTION_REG,7    ;turn on the built-in pull-up resistors for PORTB
    MOVLW   0b00000011
    MOVWF   WPUB            ;use the pull-ups on RB0 and RB1 only
    BSF     TRISB,0         ;RB0 is an input (Select button)
    BSF     TRISB,1         ;RB1 is an input (Write button)
    BCF     TRISA,0         ;RA0 is an output (selection LED)
    BCF     TRISA,1         ;RA1 is an output (main loop marker)
    BCF     TRISA,2         ;RA2 is an output (error LED)
    CLRF    TRISC           ;all of PORTC is an output (the display)
    MOVLW   249
    MOVWF   PR2             ;Timer2 counts 250 steps (this makes 1 ms)
    ;       Bank 0
    BANKSEL PORTA
    CLRF    PORTA           ;all the markers and LEDs start OFF
    CLRF    PORTC           ;display starts blank
    CLRF    ee_status       ;no read, no write, no error
    CLRF    CurrentAddress ;use EEPROM address 0x00
    MOVLW   0x41
    MOVWF   Pending         ;the pick starts out as 'A'
    CLRF    Selecting
    CLRF    TimeoutCount
    CLRF    SelDown
    CLRF    WrDown
    BCF     PIR1,1          ;clear the Timer2 flag
    MOVLW   0b01001101      ;Timer2 on, the flag is set every 10 ms
    MOVWF   T2CON
```

•Figure 12: SFRs and Setup for EEPROM R/W Routines and timed Section

```
; MAIN LOOP: this runs over and over and never waits for the EEPROM.
Main:                                                               LG
    BSF     PORTA,1         ;marker ON
    BCF     PORTA,1         ;marker OFF right away

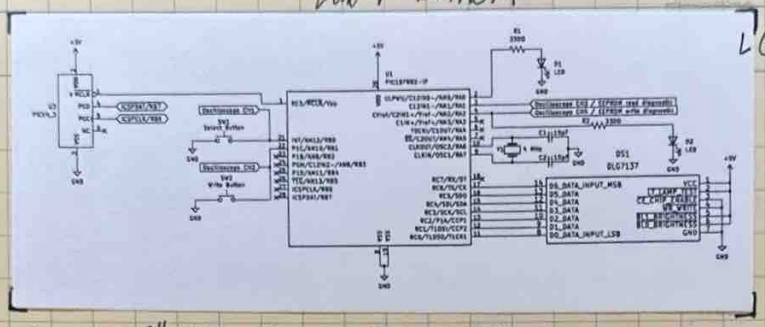
; Is a write finished?
    BTFSS   ee_status,EEPROM_WRITE ;was a write started?
    GOTO    ErrorLed        ;no
    BTFSS   PIR2,4          ;EEIF: the chip sets this when the write is done
    GOTO    ErrorLed        ;not done yet, so keep running
    CALL    EEPROMVerifyWrite ;it is done, so check it one time

; The error LED always matches the error flag
ErrorLed:
    BTFSC   ee_status,EEPROM_ERROR ;is there an error?
    GOTO    ErrorOn
    BCF     PORTA,2         ;no error, LED OFF
    GOTO    CheckTimeout
ErrorOn:
    BSF     PORTA,2         ;error, LED ON
```

•Figure 13: Main Operation code and ERROR checks

-For all code see Repo Lab9 & LG EEPROM Part2.

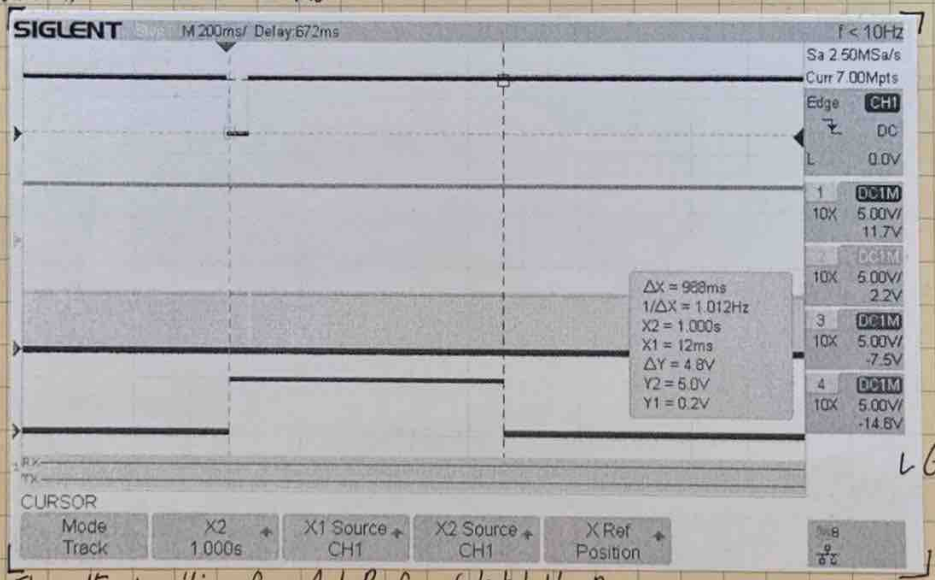
10/21/2026
LRG



LG

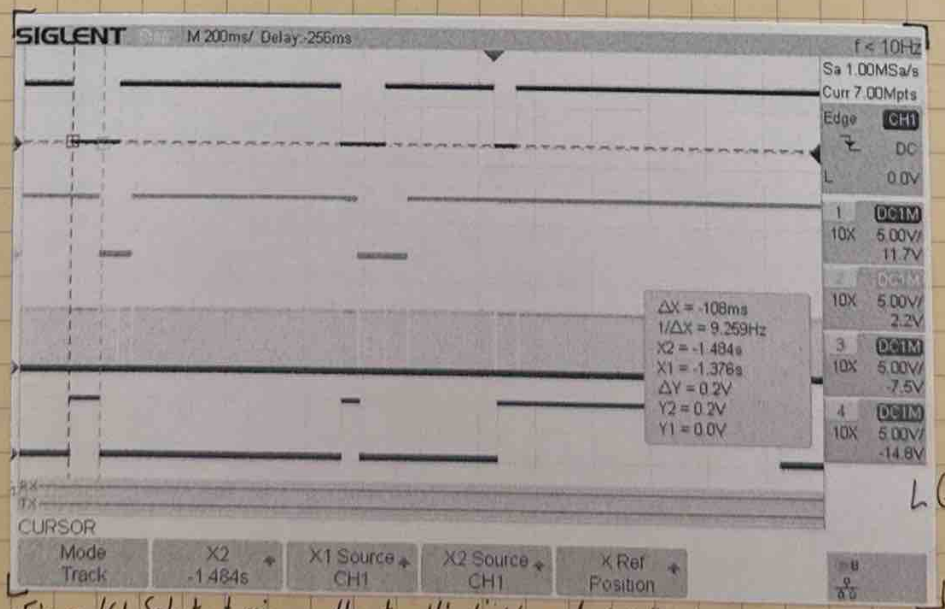
•Figure 14: Full Schematic for Part 2 EEPROM

-The next parts are of normal operation where select (SW1) can toggle and has a 1 sec led indication and if Write (SW2) is selected it will reset the count and can disable this indication transition LED on RA0.



LG

•Figure 15: transition from A to B for Select button Press



LG

•Figure 16: Select turning on the transition light and Write button resting the timer (1 sec)

Lab 9 - EEPROM

10/20/2026
LRG

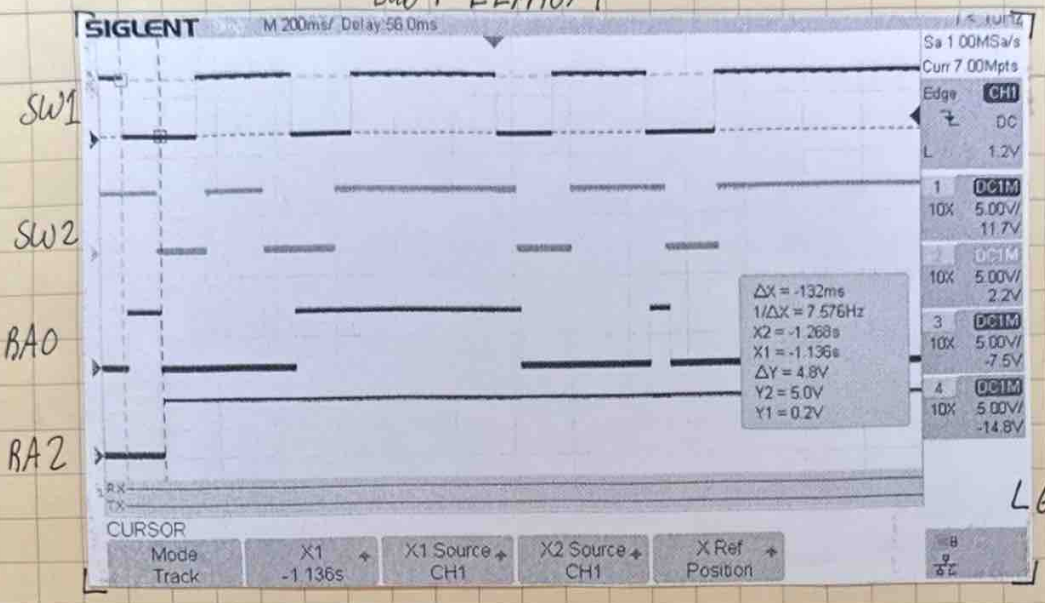
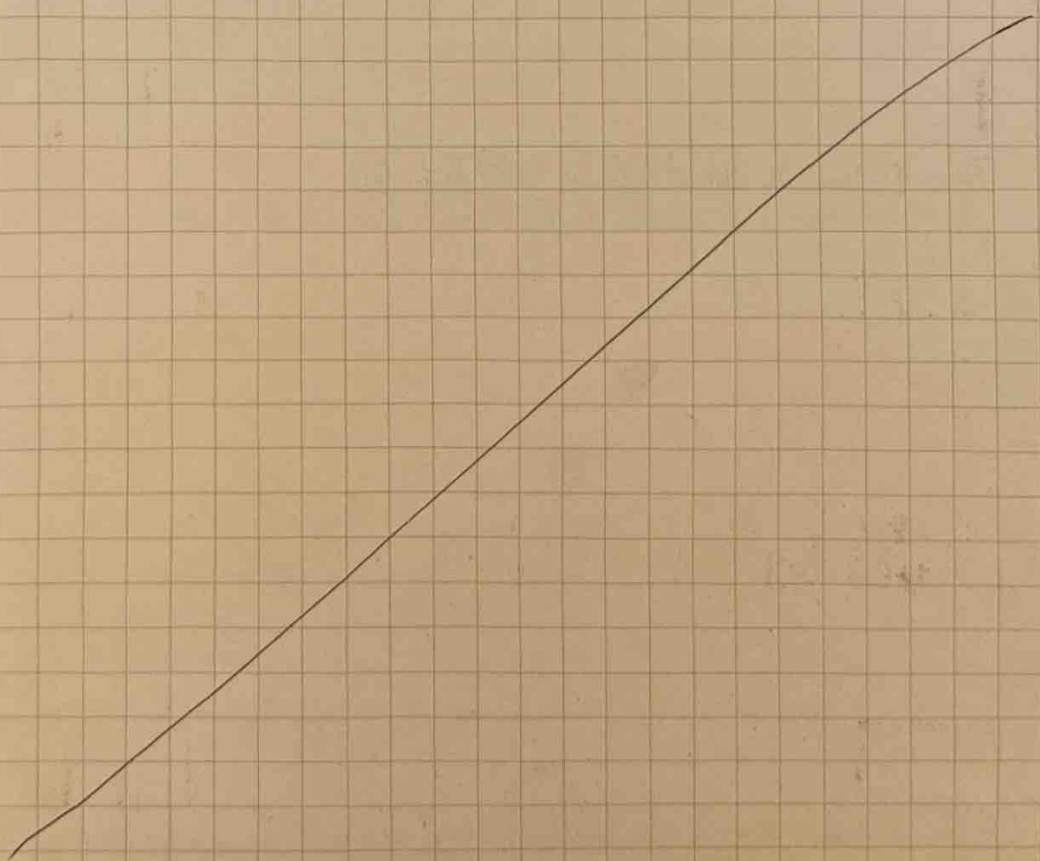


Figure 17: Proof of ERROR LED working only when code for wrong letter is written

TJR
10-22-26

-In the EEPROM Write section of code there is a section of Test code to through the error which is the only way to cause an error from the write is not sending the hex value for A or B. The purpose is control. I hold in memory

L6



10/20/2026
LRG

Lab 9 - EEPROM

•Part 3 - Four Digit Electronic Lock

•Goal: Extend Part 2 into a keypad controlled electronic lock. Enter a four digit-PIN followed by # to activate a simulated solenoid output for one second. The PIN is read from data EEPROM. Safety: Do not connect a solenoid directly to a PIC output. A real inductive load requires an appropriate driver and transient protection. Keep the simulated output inactive during reset and startup as well as normal locked operation.

•Initial PIN setup: Before running part 3, manually program a four-digit PIN into four consecutive data EEPROM addresses, with one decimal digit (0-9) in each address. Choose the address and verify the stored digits. This is a one-time setup. PIN changes are not part of Part 3, and the PIN must not appear anywhere in the program source.

•Reuse from previous labs: Use EEPROMRead from Part 2 without changing its interface. PIN entry and display handling belong to the application, not the EEPROM routines. Bring back the keypad scanner and DL6 7137 display from Lab 2. A PIN-entry routine can work much like ParseHex from the UART lab: recognize one key-press event at a time, collect digits in RAM, and track how many digits have been entered.

Key	Action
0 – 9	Add one digit to the PIN entry in RAM
*	Cancel the entry and clear the collected digits
#	Submit the entry for comparison with the stored PIN
A – D	No PIN-entry action

LG

•Table 3: Keypad behavior for the (16 button) 4x4 keypad (V1 pg 20) for hex value and more values

- Accept one digit per debounce physical key press, not one per keypad scan.
- Accept exactly four digits followed by #.
- A # press with fewer than four digits rejects and clears the entry. Reject an entry if a fifth LG fifth digit is pressed; don't accept its first four digits as a valid PIN.
- After a correct entry, drive the simulated solenoid output High for approximately one second, then LOW. An incorrect or incomplete entry must not activate it.
- Clear the collected entry after #, after *, and after reset. A user can then begin another attempt.
- Provide useful one-character progress/status feedback on the DL6 7137 display. Choose the characters

•Stored PIN and Startup: Use EEPROMRead to retrieve the four stored PIN digits. On startup or reset, keep the solenoid output off until the correct PIN Entered.

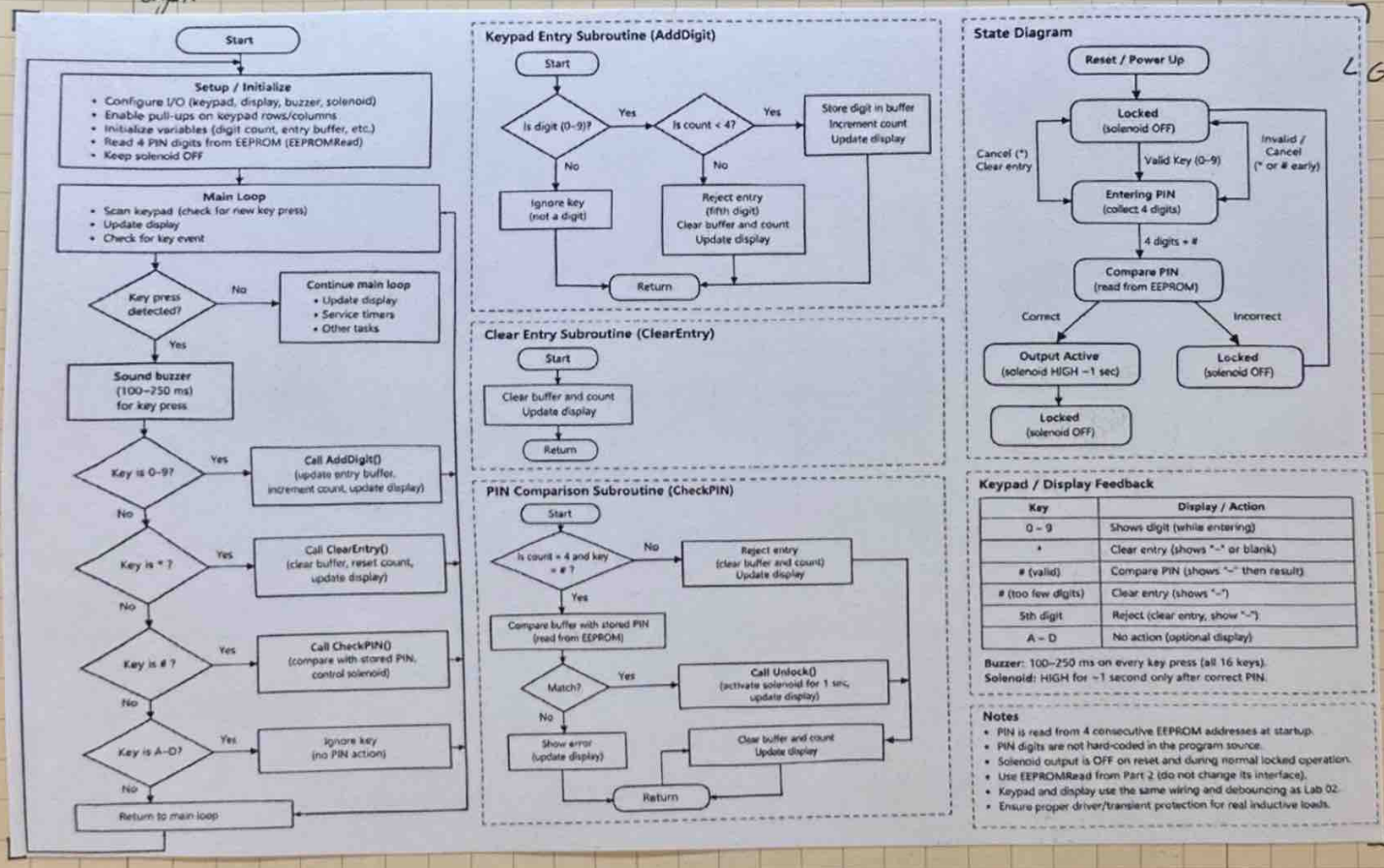
•Before Lab: Prepare or reference the keypad scanner, display wiring, and debouncing from Lab 2, including electrical precautions for simulated LG simultaneous key presses. Document the buzzer and simulated-solenoid output wiring, loading calaculations, and reset-safe output behavior. Draw a state diagram covering startup, lock operation, PIN entry, cancel, comparison, and the one-second output. Prepare flowcharts for keypad entry and PIN comparison, including *, #, incorrect-length entries, and fifth digit. Document how EEPROMRead is used to retrieve the stored PIN.

10/22/2026
LR6

Lab 4 - EEPROM

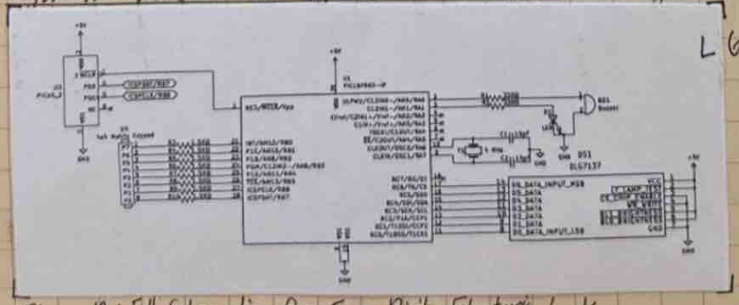
In the Lab:

1) Bring the keypad and display into the project. Verify that each debounced press produces one key event
2) Verify that the lock reads the PIN already stored in EEPROM and that the PIN digits don't appear in the source code.
3) Enter a correct PIN and confirm that the solenoid output activates only after #. Measure its one-second High pulse.
4) Test an incorrect PIN, and "#" after few than 4 digits, an fifth digit, and * during entry. None may unlock. Verify tha a new entry works after each rejected or canceled attempt.
5) Verify the buzzer responds to all 16 keys. Measure its 100-250 ms pulse and confirm the same feedback occurs for correct and incorrect PIN entries.
6) Power cycle the PIC and confirm that the solenoid output stays inactive until the saved PIN is entered again.



LR6

Figure 18: Flow chart to Four-Digit Electronic Lock and State Diagram



LR6

The resistor on buzzer is to protect the pic and buzzer is protected from current draw.

Figure 19: Full Schematic for Four Digit Electronic Lock

10/22/2026
LR6

Lab 9 - EEPROM

型 号 Type	TMB-12A01	TMB-12A03	TMB-12A05	TMB-12A12
额定电压(V) Rated Voltage	1.5	3	5	12
电压范围(V) Opcration Voltage	1.2~2.5	2.5~4	4~7	8~15
额定电流(mA) Rated Current	≤20	≤35	≤30	≤20
声压电平(dB) Sound Output	≥80	≥82	≥85	≥85
谐振频率(Hz) Resonant Freq	2300±500	2300±500	2300±500	2300±500
工作温度(℃) Operating Temp	-20~+60	-20~+60	-20~+60	-20~+60
储存温度(℃) Storage Temp	-20~+70	-20~+70	-30~+85	-30~+85
重 量(g) Weight	2	2	2	2

LG

- Measured current was 1mA well below any concern.

- Figure 20: Table of Buzzer eletrical loads

```
; TEMPORARY - saves PIN 1 2 3 4 at EEPROM 0x10-0x13.
; Delete everything from here to End after the PIN is saved.
;*****************************************************************
EEPROMWritePin:                                          LG
    MOVLW   0x10                ;digit 1
    MOVWF   ee_address
    MOVLW   0x01
    MOVWF   ee_data
    CALL    EEPROMWrite
    MOVLW   0x11                ;digit 2
    MOVWF   ee_address
    MOVLW   0x02
    MOVWF   ee_data
    CALL    EEPROMWrite
    MOVLW   0x12                ;digit 3
    MOVWF   ee_address
    MOVLW   0x03
    MOVWF   ee_data
    CALL    EEPROMWrite
    MOVLW   0x13                ;digit 4
    MOVWF   ee_address
    MOVLW   0x04
    MOVWF   ee_data
    CALL    EEPROMWrite
    RETURN
EEPROMWrite:
    BANKSEL EEADR
    MOVF    ee_address,W
    MOVWF   EEADR               ;address to write
    BANKSEL EEDATA
    MOVF    ee_data,W
    MOVWF   EEDATA              ;value to write
    BANKSEL EECON1
    BCF     EECON1,7            ;EEPGD = 0 (data EEPROM)
    BSF     EECON1,2            ;WREN = 1 (allow writing)
    MOVLW   0x55                ;required unlock sequence
    MOVWF   EECON2
    MOVLW   0xAA
    MOVWF   EECON2
    BSF     EECON1,1            ;WR = 1 (start the write)
WaitWrite:
    BTFSC   EECON1,1            ;wait until WR clears
    GOTO    WaitWrite
    BCF     EECON1,2            ;WREN = 0 (writing off again)
    BANKSEL PORTA               ;back to bank 0
    RETURN
```

- Figure 21: Added code to put in the memory to for code entry saved in EEPROM memory

-The purpose was to load this into memory so I can rerun with the saved Write and won't have it in code any more. This was removed after saved to EEPROM memory.

10/22/2026
LRG

Lab 9 - EEPROM

-The code to unlock was 1,2,3,4 and when startup happens it will first show L for "Locked", * will clear any entries and if more then 4 digits are pressed "E" will show or if # is press early. But if 1,2,3,4 is entered then # it will display "O" and the LED will turn on for 1 sec acting for the solenoid. No captures for this section because it was a the PIC16F737 display to verify. Proper operation. The program worked as desired and pushed code and got everything pushed to github and have a clean working tree.

TSR
10-23-26

```
Setup:
    ;       Bank 0
    BANKSEL PORTA
    CLRF    PORTA               ;buzzer, solenoid and LED start OFF
    CLRF    PORTB               ;keypad rows start low
    CLRF    PORTC               ;display starts blank

    ;       Bank 3
    BANKSEL ANSEL
    CLRF    ANSEL               ;make every pin a normal digital pin
    CLRF    ANSELH

    ;       Bank 1
    BANKSEL TRISA
    CLRF    TRISA               ;PORTA all outputs (RA0 buzzer, RA1 solenoid + LED)
    CLRF    TRISC               ;PORTC all outputs (display)
    MOVLW   0b11110000
    MOVWF   TRISB               ;RB0-RB3 rows out, RB4-RB7 columns in
    MOVWF   WPUB                ;weak pull-ups on the column pins
    BCF     OPTION_REG,7        ;turn weak pull-ups on (RBPU = 0)
    MOVLW   249
    MOVWF   PR2                 ;Timer2 period = 250 counts

    ;       Bank 0
    BANKSEL T2CON
    MOVLW   0b01001101          ;postscale 1:10, Timer2 ON, prescale 1:4 -> 10 ms
    MOVWF   T2CON

    CLRF    KeyNow              ;start with a clean slate after reset
    CLRF    KeyLast
    CLRF    KeyDone
    CLRF    DigitCount
    CLRF    BuzzTimer
    CLRF    LockTimer

    MOVLW   0x4C                ;'L'
    MOVWF   PORTC               ;show LOCKED
;The EE Data Memory was set so that the pin was saved during the setup and program process!
```

LG

- Figure 22: SFR's and Setup for this section

- To see the rest of the code for part 3 go to the following link. -
https://github.com/groeluke/Lab_9_EEPROM/tree/main/LAB_9_Part_3_EEPROM.X

LG

10/22/2026
LRG

Lab 9 - EEPROM

•Part 4 - Authenticate PIN Change

•Goal: Extend the Part 3 lock so that the user can set a PIN on first use and change it with the keypad. Store the PIN and its validity in EEPROM.

•PIN storage and startup: Use five consecutive EEPROM address: four for the PIN digits (0-4) and one for a validity marker. Choose and document the addresses.

- Marker = 0xCH the stored PIN is valid.
- Any other marker value: the PIN is invalid.

-Before the first Part 4 run, use the programmer to set the marker to an invalid value, such as 0xFF.

•Service modes: At startup, the stored PIN-valid marker determines whether the lock begins in invalid PIN mode or normal locked mode. The keypad must be serviced in all modes and must not be blocked for more than 20ms, except during the one-second solenoid activation and when a new PIN is written to EEPROM. The buzzer must sound for 100-250ms on every debounced key pressed LG in all modes. Holding a key must produce only one key press; another press is accepted only after the key is released and pressed again. Each PIN entry accepts only digits 0-9, followed by # after exactly four digits. Use the PL67137 to show whether the current or new PIN is being requested and whether saving succeeds or fails. Choose and document the display characters.

-Invalid PIN mode: the solenoid never activates in invalid PIN mode.

1) Enter a new four-digit PIN followed by #.
2) Enter the same four digits followed by #.

-Matching entries are saved as the new PIN. After a successful save, the lock enters normal locked mode. A mismatch, ~~imp~~ LG incomplete entry, unexpected key, or * clears the entry and restarts invalid PIN mode without saving.

- Normal locked mode:

1) Enter the current four-digit PIN followed by # to unlock.
2) To change the PIN, press C followed by #.

-A correct PIN activates the simulated solenoid for approximately one second. The lock remains in normal locked mode. An incorrect or incomplete entry, unexpected key, or * clears the entry and restarts normal locked mode. ~~An incorrect or incomplete entry, unexpected key, or~~ LG "If C is not the first key or is not followed by #, return to normal locked mode.

-PIN change mode: The solenoid never activates in PIN change mode.

1) Enter the current four digit PIN followed by #.
2) Enter a new four digit PIN followed by #.
3) Enter the new PIN again followed by #.

-If the current PIN is correct and the two new entries match, save the new PIN and return to normal locked mode. An incorrect current PIN, mismatched new PINs, incomplete entry, unexpected key, or * cancels the change and returns to normal locked mode with the old PIN unchanged. While saving a new PIN, the lock ignores keypad input and keeps the solenoid OFF. EEPROM write failures use the PIN-error behavior described on the next page.

10/22/2026
LRG

Lab 9 - EEPROM

- Reusable PIN-matching routines: Consider two application subroutines, Both can use indirect addressing with FSR and INDF for PIN digits held in RAM.

• CurrentPINMatch: Collect the keypad sequence dddd#, where each d is a digit from 0 to 9. Consider four consecutive GPR locations with a pointer or index. Store each digit and advance the pointer. The fifth key press must be #; do not store it or advance the pointer. * or any unexpected key aborts. After #, use EEPROMRead to compare the four entered digits with the stored PIN. Report whether they match. Use this routine for normal unlocking and for authenticating a PIN change; the calling code decides whether to activate the solenoid.

• NewPINMatch: Collect the keypad sequence dddd# dddd#. Consider eight consecutive GPR locations, four for each new PIN entry. Store each digit from 0 to 9 and advance the pointer. The fifth and tenth key presses must be #; don't store them or advance the pointer. * or any unexpected key aborts. After the final #, compare the two groups of four digits in RAM and report whether they match. This routine does not write EEPROM. Shared helper routines may be useful. Use RAM GPRs as needed to track key entry, state, pointer or index values, match flag and other working data. Document the GPR assignments and how each routine uses them in the lab book.

- EEPROM PIN write sequence (PINWrite):

-Create an application subroutine named PINWrite: to manage the six EEPROM write steps below. Use the Part 2 EEPROMWrite and EEPROMVerifyWrite routines without changing their interfaces. Start PINWrite only after NewPINMatch succeeds and, for a change in normal locked mode, CurrentPINMatch has authenticated the current PIN. Set a RAM PIN_UPDATE flag before the first write. While it is set, keep the solenoid off and suspend normal lock operation. PINWrite handles EEPROM write operation and verification.

LG

Step	EEPROM location	Value to write
1	PIN-valid marker	0x00 (invalid)
2	PIN digit 1	First new digit
3	PIN digit 2	Second new digit
4	PIN digit 3	Third new digit
5	PIN digit 4	Fourth new digit
6	PIN-valid marker	0xC4 (valid)

- Table 4: EEPROM PIN write sequence table.

- Complete and Verify each step before starting the next. Check EEPROM_ERROR after verification. EEPROMWrite may skip a byte that already contains the request value; confirm that the byte is correct before advancing. During PINWrite, intentionally block all other CPU work. Wait for each EEPROM write to finish, verify it, and continue to the next step. Do not resume normal operation until the sequence completes or stops with an error.

10/22/2026
LRG

• Success: After step 6 is verified, clear PIN_UPDATE and return to normal locked operation with the new PIN.

• Write or verification error: Stop the sequence. Do not write any remaining bytes. Keep the solenoid OFF, activate a PIN-error indicator on an available PORTA output, display an error character. Do not accept another PIN attempt until power is cycled.

• Power interruption: On the next startup, check the marker normally. If it is invalid, required two matching PIN entries before trying PINWrite again.

Before Lab:

-Update the Part 3 state diagram to show invalid PIN mode, normal locked mode, PIN change mode, PINWrite success, and error handling. Include the current PIN Match and New PIN Match flows. Draw a flowchart for PINWrite showing all six EEPROM steps and the verification/error decisions. Document the five EEPROM addresses, PIN_UPDATE, display characters, and the PORTA error indicator with its loading calculation.

In the Lab:

1) Program an invalid PIN marker. Verify that Verify LG first-run setup requires two matching PIN entries. Test *, incomplete entries, mismatched confirmation, and long key holds without EEPROM writes or repeated key presses.

2) Complete setup, reset the PIC, and unlock using the stored PIN. Measure the keypad servicing interval during normal operation and verify that it does not exceed 20ms.

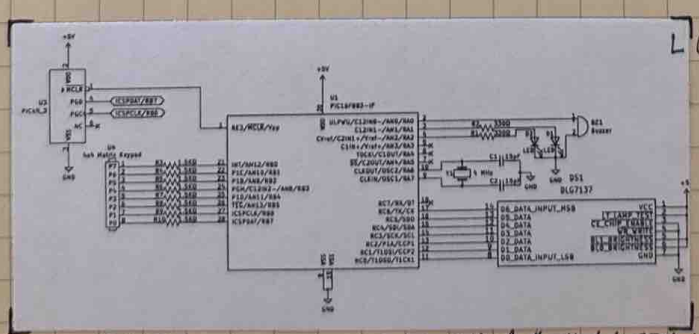
3) Verify that C alone does not enter PIN change mode. Press C followed by # and test an incorrect current PIN, cancellation, and mismatch new PIN configuration. Verify that the stored PIN remains unchanged.

4) Change the PIN successfully. Use the debugger to confirm the six PINWrite steps occur in order, with each step verified before the next. Confirm the old PIN fails and the new PIN works after a power cycle.

5) While PIN_UPDATE is set, verify that keypad processes cannot unlock or cancel the EEPROM PIN write sequence.

6) Interrupt the sequence after the marker becomes invalid. Power cycle and verify that first-run setup is required. Simulate a verification error and demonstrate the PORTA and display error indicators, inactive solenoid, and required power cycle before another attempt.

LG



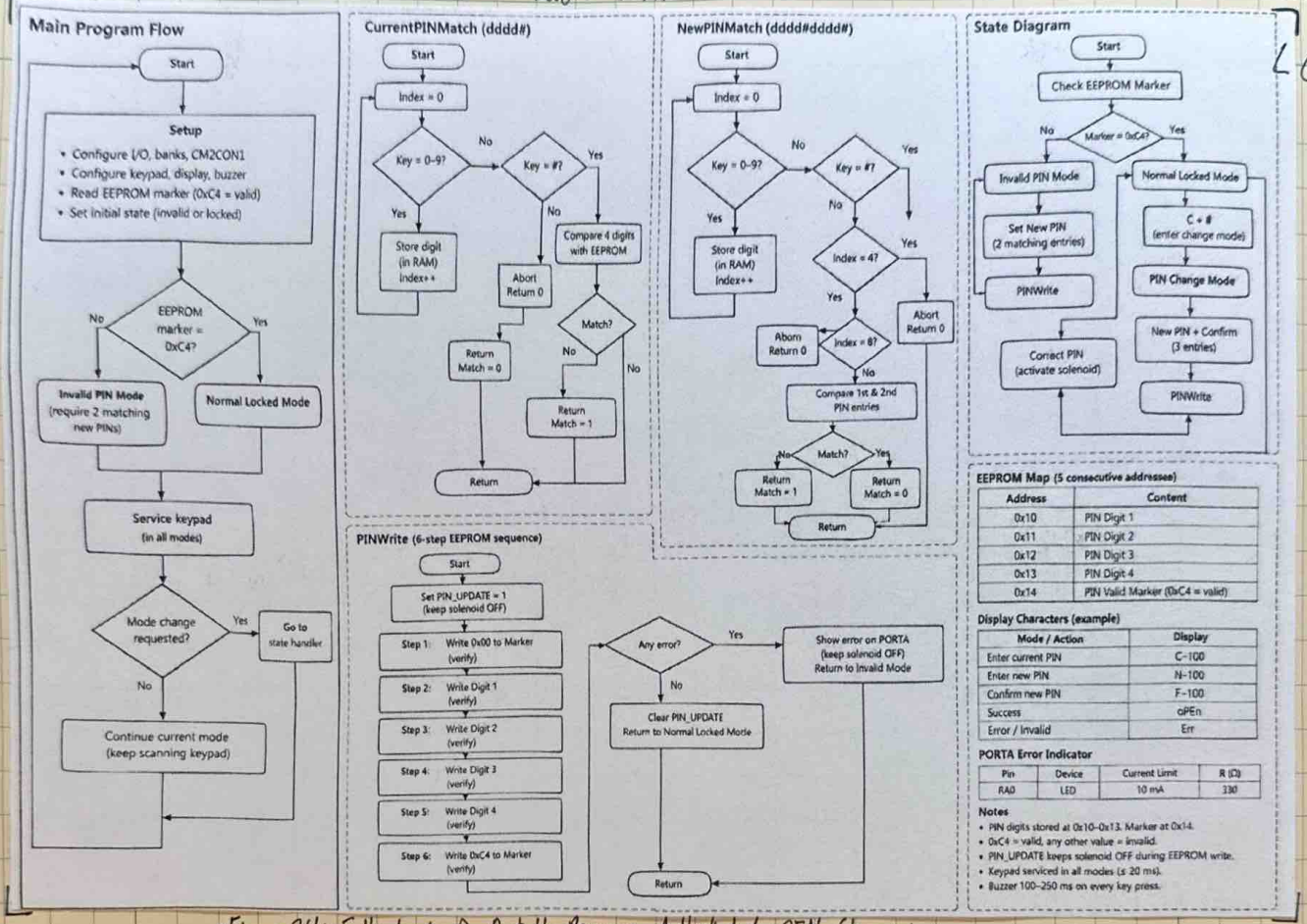
• Figure 23: Full working schematic for Part 4 Authenticated PIN change

-The RA2 LED is for the PIN error LED and the display letters go as follows:

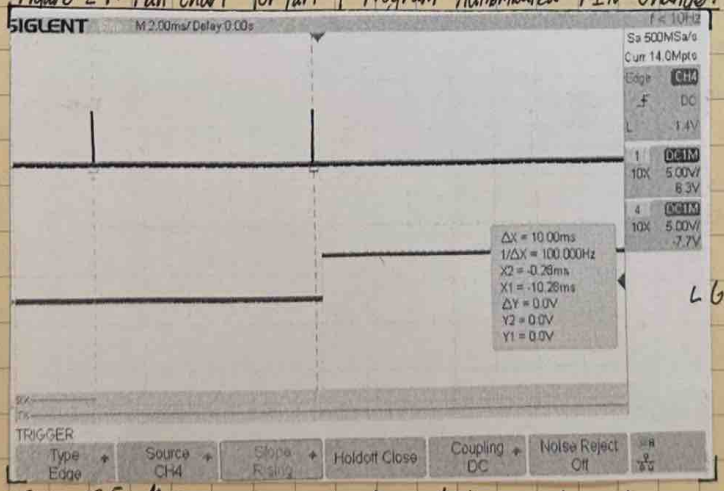
- L = Locked
- R = re-enter the new PIN
- 1-4 = digits typed
- 0 = open
- F = save failed
- N = enter NEW PIN
- C = enter current PIN (change Pin)
- E = entry rejected
- S = new PIN saved

10/23/2026
LRG

Lab 4 - EEPROM

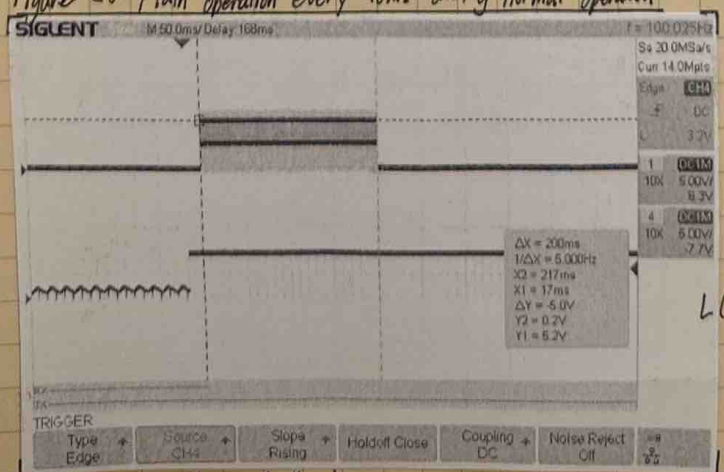


•Figure 24: Full chart for Part 4 Program Authenticated PIN Change:



LG

•Figure 25: Main operation every 10ms during normal operation



LG

•Figure 26: the time of the buzzer is operating when button is pressed.

LG
10/23/2026
LRG

```
; EEPROM routine variables
   ee_address      EQU 0x70   ;EEPROM address to read or write
   ee_data         EQU 0x71   ;byte read, or byte to write
   ee_status       EQU 0x72   ;status flags
   EEPROM_READ     EQU 0      ;bit 0 - a read is happening
   EEPROM_WRITE    EQU 1      ;bit 1 - a write started, not checked yet
   EEPROM_ERROR    EQU 2      ;bit 2 - the last write did not check out
   EE_GIE_SAVED    EQU 7      ;bit 7 - remembers GIE during a write

; Application variables
   KeyNow          EQU 0x73   ;key down now (ASCII, 0 = none)
   KeyLast         EQU 0x74   ;key down on the last scan
   KeyDone         EQU 0x75   ;key already counted
   Count           EQU 0x76   ;digits typed in this group (0-4)
   BuzzTimer       EQU 0x77   ;buzzer time left (10 ms units)
   LockTimer       EQU 0x78   ;solenoid time left (10 ms units)
   Temp            EQU 0x79   ;loop counter
   Mode            EQU 0x7A   ;which mode we are in (see below)
   Base            EQU 0x7B   ;0 = first group, 4 = second group
   Flags           EQU 0x7C   ;program flags (see below)

; Flags bits
   PIN_UPDATE      EQU 0      ;1 = PINWrite is running
   PIN_ERROR       EQU 1      ;1 = save failed, locked until power cycle

; Modes
   MODE_NORMAL     EQU 0      ;normal locked mode
   MODE_NORMAL_C   EQU 1      ;normal mode, C was pressed, waiting for #
   MODE_INVALID    EQU 2      ;invalid PIN mode (set a new PIN)
   MODE_CHANGE_CUR EQU 3      ;PIN change: enter current PIN
   MODE_CHANGE_NEW EQU 4      ;PIN change: enter new PIN twice

; Bank 0 keep these 8 in a row
   Entry0          EQU 0x22   ;group 1, digit 1
   Entry1          EQU 0x23
   Entry2          EQU 0x24
   Entry3          EQU 0x25   ;group 1, digit 4
   Entry4          EQU 0x26   ;group 2, digit 1
   Entry5          EQU 0x27
   Entry6          EQU 0x28
   Entry7          EQU 0x29   ;group 2, digit 4

; Settings
   PinBase         EQU 0x10   ;EEPROM address of PIN digit 1
   MarkerAddr      EQU 0x14   ;EEPROM address of the valid marker
   MarkerValid     EQU 0xC4   ;marker value that means "PIN is valid"
   BeepTicks       EQU 20     ;20 x 10 ms = 200 ms
   OpenTicks       EQU 100    ;100 x 10 ms = 1 second
```

LG

•Figure 27: Required GPR's to get the code working as desired

-The GPR's all serve a purpose whether is holding button selection to saving EEPROM data on settings. For the rest of the code see link below:

-https://github.com/groelwke/Lab-9-EEPROM/tree/main/LAB_9_Part_4_EEPROM.X

LG

Lab 9 - EEPROM

10/23/2026
LRG

```
Setup:
    ;       Bank 3
    BANKSEL ANSEL
    CLRF    ANSEL               ;make every pin a normal digital pin
    CLRF    ANSELH
    ;       Bank 1
    BANKSEL TRISA
    CLRF    TRISA               ;PORTA all outputs (RA0 buzzer, RA1 solenoid + LED)
    CLRF    TRISC               ;PORTC all outputs (display)
    MOVLW   0b111110000
    MOVWF   TRISB               ;RB0-RB3 rows out, RB4-RB7 columns in
    MOVWF   WPUB                ;weak pull-ups on the column pins
    BCF     OPTION_REG,7        ;turn weak pull-ups on (RBPU = 0)
    MOVLW   249
    MOVWF   PR2                 ;Timer2 period = 250 counts
    ;       Bank 0
    BANKSEL PORTA
    CLRF    PORTA               ;buzzer, solenoid and LED start OFF
    CLRF    PORTB               ;keypad rows start low
    CLRF    PORTC               ;display starts blank
    BANKSEL T2CON
    MOVLW   0b01001101          ;postscale 1:10, Timer2 ON, prescale 1:4 -> 10 ms
    MOVWF   T2CON
    CLRF    KeyNow              ;start with a clean slate after reset
    CLRF    KeyLast
    CLRF    KeyDone
    CLRF    Count
    CLRF    Base
    CLRF    BuzzTimer
    CLRF    LockTimer
    CLRF    Flags
    CLRF    Mode
    CLRF    ee_status
    MOVLW   MarkerAddr
    MOVWF   ee_address
    CALL    EEPROMRead          ;marker -> ee_data
    MOVF    ee_data,W
    XORLW   MarkerValid         ;is it 0xC4?
    BTFSS   STATUS,2            ;read the valid marker to pick the starting mode
    GOTO    StartInvalid        ;no - PIN is not valid
    MOVLW   MODE_NORMAL         ;yes - normal locked mode
    MOVWF   Mode
    MOVLW   0x4C                ;'L'
    MOVWF   PORTC
    GOTO    Main
StartInvalid:
    MOVLW   MODE_INVALID        ;invalid PIN mode - ask for a new PIN
    MOVWF   Mode
    MOVLW   0x4E                ;'N'
    MOVWF   PORTC
```

LG

•Figure 28: SFR's for setting up the PIN Change code

-In the EEPROMWrite the commented section (MOVLW 1 & ADDWF EEDATA,F) will cause the error when a new password is attempting to be rewritten. EEPROMWrite happens when a new code trys to be entered. The lock stores the 4 digit PIN in EEPROM (0x10, 0x11, 0x12, 0x13) with the marker byte at (0x14). At power up it reads the marker. If it is 0xC4, the lock starts in normal mode. Other wise it starts in invalid PIN mode and asks for a new PIN 2 times. In normal mode, C the ## starts a PIN change, which needs the current PIN and then the new PIN 2 times until you see S (Save). Then enter again to verify seeing O (open) and L (Locked).

LG

10/23/2026
LRG

LG

```
; HandleKey runs once per key press (the key is in KeyDone)
HandleKey:
    BSF     PORTA,0             ;buzzer ON for every key
    MOVLW   BeepTicks
    MOVWF   BuzzTimer           ;RunTimers turns it off after 200 ms

    BTFSC   Flags,PIN_ERROR     ;save failed so ignore keys until power cycle
    RETURN
    BTFSC   Flags,PIN_UPDATE    ;PIN being saved so ignore keys
    RETURN
    MOVF    LockTimer,F         ;is the solenoid on right now?
    BTFSS   STATUS,2
    RETURN                      ;yes beep only so ignore the key

    MOVF    Mode,W
    BTFSC   STATUS,2            ;Mode 0
    GOTO    NormalKey
    XORLW   MODE_NORMAL_C
    BTFSC   STATUS,2
    GOTO    NormalCKey
    MOVF    Mode,W
    XORLW   MODE_INVALID
    BTFSC   STATUS,2
    GOTO    InvalidKey
    MOVF    Mode,W
    XORLW   MODE_CHANGE_CUR
    BTFSC   STATUS,2
    GOTO    ChangeCurKey
    GOTO    ChangeNewKey        ;must be MODE_CHANGE_NEW

; MODE: NORMAL LOCKED
NormalKey:
    MOVF    Count,F             ;no digits typed yet?
    BTFSS   STATUS,2
    GOTO    NormalDigits        ;already typing - not a C request
    MOVF    KeyDone,W
    XORLW   0x43                ;is the first key 'C'?
    BTFSS   STATUS,2
    GOTO    NormalDigits
    MOVLW   MODE_NORMAL_C
    MOVWF   Mode                ;C pressed - now wait for #
    RETURN

NormalDigits:
    CALL    CurrentPINMatch     ;0 = typing, 1 = PIN correct, 2 = rejected
    ADDLW   0
    BTFSC   STATUS,2
    RETURN                      ;still typing
    XORLW   1
    BTFSS   STATUS,2
    GOTO    RejectToNormal      ;wrong PIN or bad key

    CLRF    Count               ;PIN correct - unlock
    BSF     PORTA,1             ;solenoid and LED ON
    MOVLW   OpenTicks
    MOVWF   LockTimer           ;RunTimers turns it off after 1 second
    MOVLW   0x4F                ;'O'
    MOVWF   PORTC
    RETURN

NormalCKey:                     ;C was pressed, this key must be #
    MOVF    KeyDone,W
    XORLW   0x23                ;is it '#'?
    BTFSS   STATUS,2
    GOTO    BackToNormal        ;no - back to normal locked mode
    MOVLW   MODE_CHANGE_CUR     ;yes - start the PIN change
    MOVWF   Mode
    CLRF    Count
    CLRF    Base
    MOVLW   0x43                ;'C' = enter CURRENT PIN
    MOVWF   PORTC
    RETURN
```

*Figure 29: The keyhandler and all its checks for pressed keys

```
; MODE: INVALID PIN (set a new PIN: dddd# dddd#)
InvalidKey:
    CALL    NewPINMatch         ;0 = typing, 1 = both entries match, 2 = rejected
    ADDLW   0
    BTFSC   STATUS,2
    RETURN
    XORLW   1
    BTFSS   STATUS,2
    GOTO    InvalidReject
    CALL    PINWrite            ;save the new PIN (blocks until done)
    RETURN

InvalidReject:                  ;clear the entry and start over
    CLRF    Count
    CLRF    Base
    MOVLW   0x45                ;'E'
    MOVWF   PORTC
    RETURN

; MODE: PIN CHANGE - step 1, enter the current PIN
ChangeCurKey:
    CALL    CurrentPINMatch
    ADDLW   0
    BTFSC   STATUS,2
    RETURN
    XORLW   1
    BTFSS   STATUS,2
    GOTO    RejectToNormal      ;wrong current PIN - cancel the change
    MOVLW   MODE_CHANGE_NEW     ;current PIN was right
    MOVWF   Mode
    CLRF    Count
    CLRF    Base
    MOVLW   0x4E                ;'N' = enter NEW PIN
    MOVWF   PORTC
    RETURN

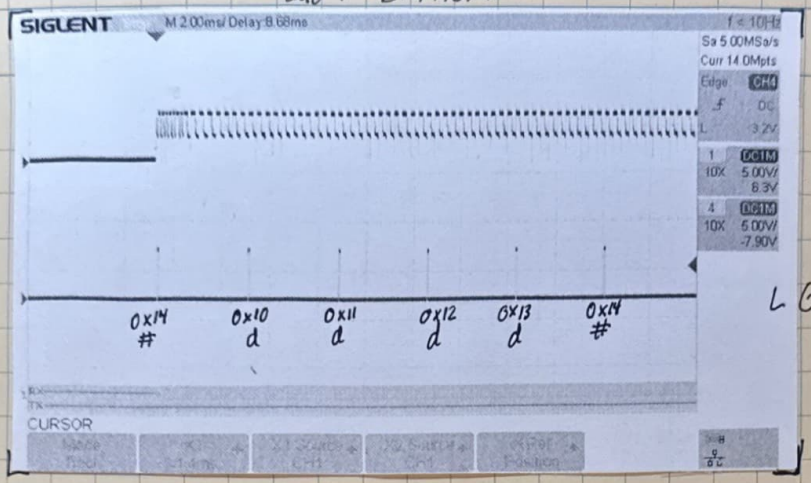
; MODE: PIN CHANGE - step 2, enter the new PIN twice
ChangeNewKey:
    CALL    NewPINMatch
    ADDLW   0
    BTFSC   STATUS,2
    RETURN
    XORLW   1
    BTFSS   STATUS,2
    GOTO    RejectToNormal      ;entries did not match - cancel the change
    CALL    PINWrite            ;save the new PIN (blocks until done)
    RETURN
```

Figure 30: Change code or invalid key pressed handlers

```
;EEPROMWrite: in = ee_address and ee_data. Starts the write and returns.
;Does nothing if a write is already going, or if the stored byte is
;already the same.
EEPROMWrite:
    BTFSC   ee_status,EEPROM_WRITE ;is a write already going?
    RETURN
    BANKSEL PIR2
    BTFSC   PIR2,4          ;is EEIF still set from an old write?
    RETURN
    CALL    EEReadByte      ;W = the byte stored right now
    XORWF   ee_data,W       ;zero if it is the same as what we want
    BTFSC   STATUS,2
    RETURN                  ;same value, so do not write
    BSF     ee_status,EEPROM_WRITE ;a real write is starting
    BANKSEL EEADR
    MOVF    ee_address,W
    MOVWF   EEADR           ;the address to write
    MOVF    ee_data,W
    MOVWF   EEDATA          ;the byte to write
    ;MOVLW   1              ;TEST ONLY: remove the ; on these two lines to write
    ;ADDWF   EEDATA,F       ;the WRONG byte, so the check fails on purpose
    BANKSEL EECON1
    BCF     EECON1,7        ;pick data EEPROM
    BSF     EECON1,2        ;allow writing (WREN)

    BCF     ee_status,EE_GIE_SAVED ;assume interrupts were OFF
    BTFSC   INTCON,7        ;were they ON?
    BSF     ee_status,EE_GIE_SAVED ;yes, remember that
GieOff:
    BCF     INTCON,7        ;turn interrupts OFF
    BTFSC   INTCON,7        ;make sure they are really OFF
    GOTO    GieOff
    MOVLW   0x55            ;the unlock code, in this exact order
    MOVWF   EECON2
    MOVLW   0xAA
    MOVWF   EECON2
    BSF     EECON1,1        ;start the write (we do not wait)
    BTFSC   ee_status,EE_GIE_SAVED ;were interrupts ON when we came in?
    BSF     INTCON,7        ;yes, turn them back ON
    BCF     ee_status,EE_GIE_SAVED ;done with that memory bit
    BANKSEL PORTA           ;back to bank 0
    RETURN
```

Figure 31: EEPROM Write Code to write to EEPROM Data memory



LG

•Figure 32: The EEPROM Write captured during a new saved password.

-Each time Write Step occurs EEPROM Write runs your diagnostic pin goes high and the low, so you see one pulse per step.

TJR
10-23-26

-Conclusion: Here in Lab 9 we were ask to learn how to read and write stored data within EEPROM, Verifying that even when power cycles occur it is able to retain that memory. We also were able to able reliable nonvolatile storage in embedded systems. In the Lab we were asked in part 1 to just prove we could select between A or B in RAM and save it to data EEPROM address 0x00 and then display it on the DL67137 Proving that during a power cycle we can hold/retain said value during it. By using a select and write buttons to operate.(Pg 1) Part 2 was doing the same but doing non blocking write and a separate verification routine. By using EEPROM GPR 3 for specil operations based on the desired application. But the purpose was to not wait for things to happen we go check and return.(Pg 7) Part 3 was bring Lab 2 4x4 matrix code for reading each button pressed and entering a code that was saved to memory. Where a four-digit PIN followed by # to activate a "solenoid" LED output for one second. We had to initialize it at first to save to the Kode to activate. (pg 13) Part 4 was making an entry code to be changed having the DL67137 Display Letters to help knowwhats happening in the operation. And to change the keypad entry will have a new entry code and it is saved to the EEPROM data. By using EEPROM Read, EEPROM Write, and EEPROM Verify Write ~~routines without change~~ CC to rewrite the PIN. The lesson learned was to check the actual EEPROM contents in MPLab EE DATA Memory. To verify that it contains the data where it is need.

LG